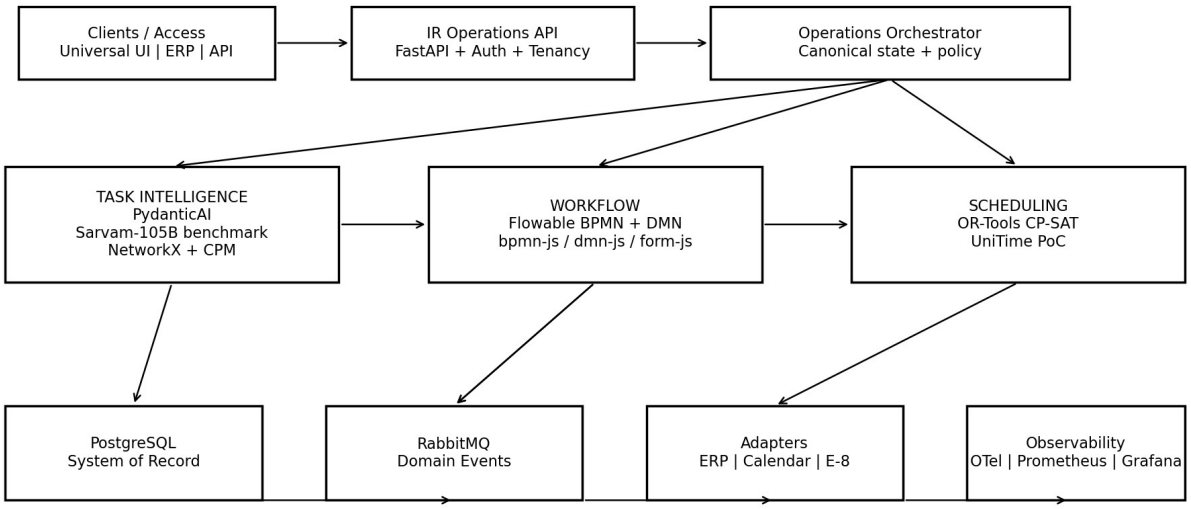
IR INFOTECH

E-7 OPERATIONS ENGINE

Complete Implementation Architecture, Open-Source Integration & Step-by-Step Delivery Plan

Unified from Task Intelligence Engine (2) + Workflow Engine (13) + Scheduling Engine (24)

Version 2.0 | 15 September 2026 | Implementation-Ready Master Plan



Principle: AI proposes; deterministic engines validate and execute; IR owns the canonical orchestration layer.

Document purpose: This is the implementation master document. It reconciles all supplied IR source documents and the approved open-source research into one non-contradictory build plan. Functional requirements remain governed by the IR source set; the later OSS research refines implementation choices without changing product behavior.

1. Document Control, Source of Truth and Reconciliation Rules

This document deliberately separates product truth from implementation choice so older candidate technologies do not conflict with the later open-source research. The source documents remain authoritative for scope and acceptance; the OSS research is authoritative for the current implementation selection.

Ref	Source	Authority in this master plan
S01	IR MAIN DOC(10)	Mandatory market-first research process, 9-dimension evaluation framework, platform goals, implementation lifecycle, original Engines 2/13/24.
S02	Consolidated Engine Directory(5)	Formal E-7 merge; Plan -> Automate -> Schedule unifying logic; consolidated outputs.
S03	Engine Architecture - IR Infotech AI Ecosystem(4)	Open-source-first philosophy, intelligence-layer boundary, platform-agnostic design, dual access modes, deterministic scheduling.
S04	2task_intelligence(1)	Task Intelligence features, deliverables, expected outcomes and acceptance criteria.
S05	13workflow(1)	Workflow features, deliverables, expected outcomes and acceptance criteria.
S06	19scheduling(1)	Scheduling features, deliverables, expected outcomes and acceptance criteria.
S07	E-7 Comprehensive Implementation Plan	Reasoning/execution separation and initial candidate stack.
S08	E-7 Complete Specification (latest supplied)	Reconciled product architecture, canonical model, APIs/events, security, test strategy, roadmap and acceptance criteria.
S09	E-7 Open-Source Research Report (latest supplied)	Current OSS adoption decisions, licensing screen, exact project roles, PoC sequence and source-code links.

1.1 Precedence rules used to prevent contradictions

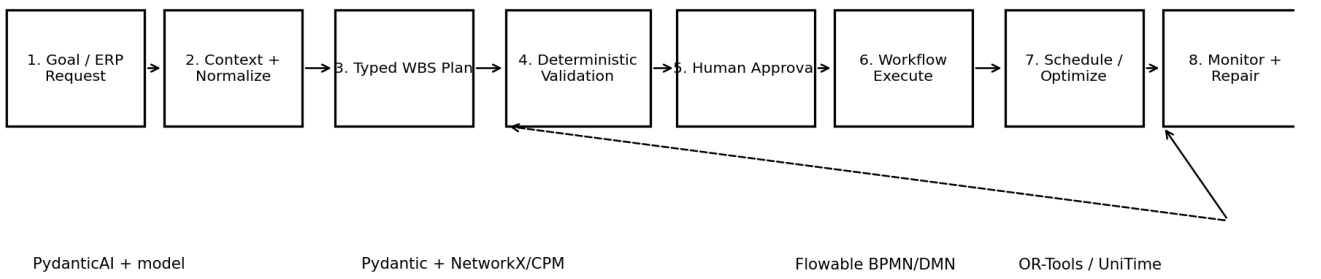
- Functional behavior and acceptance criteria: preserve S01-S08. S09 cannot remove a requirement; it only chooses how to implement it.
- Architecture invariants: API-first, intelligence-layer-only, dual access, tenant-aware, event-driven, auditable, LLM-not-authoritative, deterministic workflow/scheduling.
- Open-source implementation choices: use S09 when it refines an older candidate recommendation in S07/S08.
- Example reconciliation - AI orchestration: S08 suggested LangGraph; S09 selects PydanticAI for the MVP and keeps LangGraph conditional for later multi-agent complexity. No functional requirement changes.
- Example reconciliation - rules: S08 listed Drools/Easy Rules; S09 selects Flowable DMN + dmn-js first and keeps extra rule engines unnecessary until a measured gap appears.

- Example reconciliation - education scheduling: S08 mentioned FET/OR-Tools; S09 adds UniTime as the primary education-specific PoC while retaining OR-Tools as the generic primary solver and FET as a benchmark/fallback.
- Example reconciliation - cache: earlier notes mention Redis; the open-source research prefers PostgreSQL coordination first and Valkey only after measured need.
- Conditional components are not production dependencies until their explicit gate is met and an Architecture Decision Record (ADR) approves them.

2. Executive Implementation Summary

E-7 is the orchestration layer for work, people, time and institutional processes. It converts an operational goal into a validated plan, drives the plan through deterministic business workflows, computes feasible/optimized time and resource assignments, monitors execution, and reacts to disruptions through an auditable event loop.

Final build direction: Build a thin IR-owned canonical operations platform over proven open-source runtimes. IR owns the intelligence, domain model, cross-mode orchestration, policies, adapters and audit/explainability. Flowable owns BPMN/DMN execution; OR-Tools owns general constraint optimization; UniTime is evaluated for education scheduling; PydanticAI structures model reasoning; NetworkX validates dependencies; PostgreSQL remains the system of record; RabbitMQ is the initial broker.



Event loop: task/workflow/resource/calendar changes can trigger risk recomputation and targeted schedule repair.

3. Product Scope and Non-Goals

3.1 Mandatory capabilities

- Goal intake from natural language or structured requests with tenant, user role, deadline, constraints, resources and active organizational context.
- Hierarchical WBS generation, priorities, dependencies, effort estimates, milestones, resource recommendations, progress and blocker/risk monitoring.
- No-code BPMN-style workflow authoring, human/system tasks, single/sequential/parallel/quorum approvals, deterministic rules, SLA timers, escalation, durable execution and audit.
- Conflict-free scheduling for time, people, rooms and equipment with hard/soft constraints, optimization objectives, publish/lock semantics, incremental repair and explicit infeasibility explanations.
- Calendar synchronization, ERP adapters, versioned domain events, notifications via E-8 or external adapters, analytics and complete operational audit history.
- Universal mode and Embedded/Integrated mode using the same services but different tenant/context scopes.

3.2 Explicit non-goals

- E-7 is not a replacement ERP, LMS, HRMS, calendar suite or general project-management product.
- LLM output never directly performs an approval, changes authoritative workflow state, runs an SLA clock, bypasses authorization or becomes the scheduling solver.
- E-7 does not copy GPL/AGPL reference-product code into the proprietary core merely to reproduce standard project-management UI.
- E-7 does not deploy every researched tool. The MVP intentionally uses the minimum stack and adds conditional services only after evidence.

4. Final Open-Source Stack and Component Ownership

Layer	Project / technology	Decision	Exact E-7 role
API/orchestration	FastAPI + Pydantic	IR-owned	Canonical API, auth/tenant context, orchestration, idempotency, policies.
Task Intelligence	PydanticAI + Sarvam-105B benchmark	Primary	Typed PlanSchema generation, tools, bounded repair. Model remains replaceable.
Dependencies/CPM	NetworkX + IR CPM logic	Adopt	Cycle detection, topological order, graph traversal; IR implements business CPM/slack policy.
Workflow runtime	Flowable Engine	Primary	BPMN/CMMN/DMN runtime, human tasks, timers, persisted state, history.
Workflow authoring	bpmn-js + dmn-js + form-js	Adopt	Embedded editors; IR owns UX, permissions, versioning and deployment governance.
Rules	Flowable DMN	Primary first choice	Deterministic typed decisions; add Drools only if a proven gap exists.
General scheduling	Google OR-Tools CP-SAT	Primary	Hard/soft constraint optimization, resource assignment, repair objective.
Education scheduling	UniTime	Primary PoC	Compare against OR-Tools-native model on same school/exam dataset before production selection.
Durable code subflows	Temporal	Conditional	Add only if code-heavy long-running workflows are awkward in Flowable service tasks.
Alternative solver	Timefold Solver	Conditional	Use if Java domain/constraint modeling materially simplifies maintainability.
Advanced agent graph	LangGraph	Conditional	Add when Task Intelligence becomes persistent multi-agent/critic graph.
Education benchmark	FET 7.10.5	Benchmark/fallback	Rich timetable constraints; AGPL/legal and service-wrapper review required.

Layer	Project / technology	Decision	Exact E-7 role
Workflow fallback	Operaton	Conditional fallback	Maintain adapter abstraction; do not support dual runtimes in MVP.
System of record	PostgreSQL	Primary	Canonical state, transactional writes, outbox, audit.
Message broker	RabbitMQ	Primary initial	Domain events and asynchronous jobs; Kafka only if platform standard/scale requires it.
Cache/coordination	Valkey	Optional	Add only after measured need; start with DB + advisory locks where sufficient.
Observability	OpenTelemetry + Prometheus/Grafana	Primary pattern	Metrics/traces/log correlation; organization equivalents acceptable.
Reference integrations	OpenProject / Plane / LibrePlan / Odoo / ERPNext	Reference/adapters	Study semantics and integrate externally; never make them E-7 system of record.

4.1 OSS source-code directory

Flowable Engine: https://github.com/flowable/flowable-engine

bpmn-js: https://github.com/bpmn-io/bpmn-js

dmn-js: https://github.com/bpmn-io/dmn-js

form-js: https://github.com/bpmn-io/form-js

Google OR-Tools: https://github.com/google/or-tools

UniTime: https://github.com/UniTime/unitime

PydanticAI: https://github.com/pydantic/pydantic-ai

NetworkX: https://github.com/networkx/networkx

Temporal: https://github.com/temporalio/temporal

Timefold Solver: https://github.com/TimefoldAI/timefold-solver

LangGraph: https://github.com/langchain-ai/langgraph

FET source archive: https://lalescu.ro/liviu/fet/download.html

Operaton: https://github.com/operaton/operaton

OpenProject: https://github.com/opf/openproject

LibrePlan: https://github.com/LibrePlan/libreplan

Plane: https://github.com/makeplane/plane

Odoo: https://github.com/odoo/odoo

ERPNext: https://github.com/frappe/erpnext

PostgreSQL mirror: https://github.com/postgres/postgres

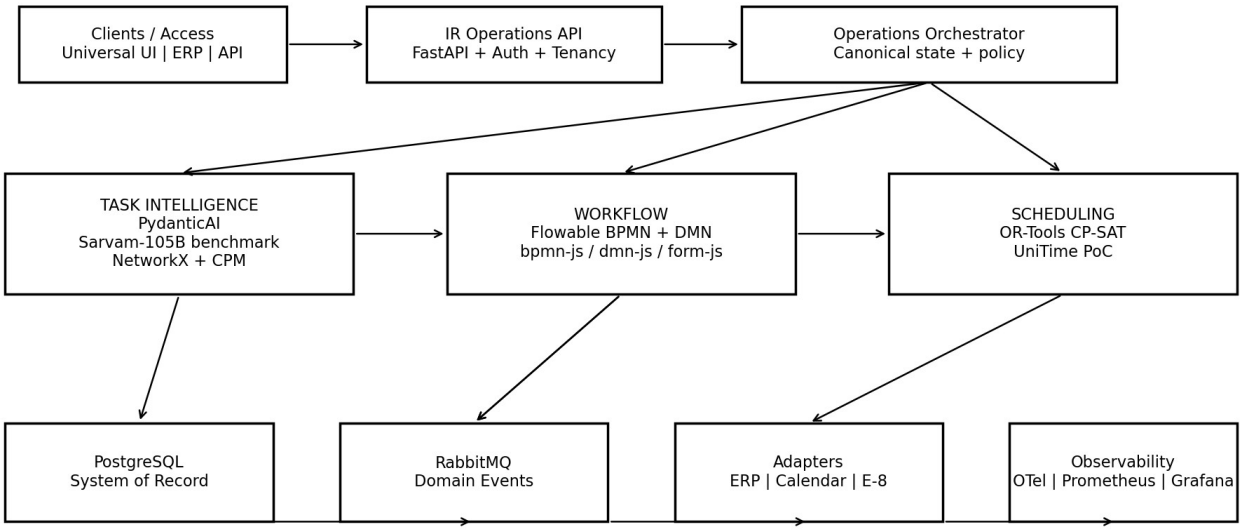
RabbitMQ: https://github.com/rabbitmq/rabbitmq-server

Apache Kafka: https://github.com/apache/kafka

Valkey: https://github.com/valkey-io/valkey

Sarvam-105B model: https://huggingface.co/sarvamai/sarvam-105b

5. Target System Architecture



Principle: AI proposes; deterministic engines validate and execute; IR owns the canonical orchestration layer.

5.1 Service boundaries

Service	Runtime	Responsibility
api-service	FastAPI	REST/OpenAPI, auth, tenant resolution, correlation IDs, idempotency keys, request validation.
plan-service	Python	Goals, plans, tasks, milestones, versioning, approval, progress and risk-facing APIs.
task-intelligence-worker	Python	PydanticAI model calls, PlanSchema generation, repair and model evaluation metadata.
dependency-service	Python	NetworkX graph validation, cycle path, topological order, CPM/slack and critical-path calculations.
workflow-adapter	Python + Flowable Java service	Deploy/version BPMN/DMN, start process, map user tasks, complete actions, query history.
scheduling-worker	Python	OR-Tools model building, solve jobs, objective scoring, infeasibility diagnostics, repair.
unitime-adapter	Python/Java boundary	Translate canonical academic data to/from UniTime during PoC/production if selected.

Service	Runtime	Responsibility
resource-service	Python	People/room/equipment registry, skills, capacity, availability, external IDs and freshness.
calendar-adapter	Python	iCal/CalDAV/provider mapping, external IDs, timezone normalization, idempotent sync.
event-gateway	Python	Transactional outbox publisher, RabbitMQ consumers, schema versions, inbox/deduplication.
audit-service	Python/PostgreSQL	Append-only logical audit with actor/source/version/correlation and before/after state.
analytics-api	Python	Planning, workflow, scheduling, utilization and reliability KPIs.
operations-ui	Web app	Plan review, workflow designer, rules/forms, task inbox, scheduler, audit/analytics dashboards.

5.2 Authority and state ownership

Domain	Authoritative owner	Rule
Goal/plan/task baseline	PostgreSQL / Plan Service	LLM proposes; approved canonical record is authoritative.
Workflow execution state	Flowable	E-7 stores canonical mapping and audit mirror; Flowable executes transitions.
DMN rule result	Flowable DMN + rule version	Deterministic result stored with inputs/version.
Schedule job/result	PostgreSQL + selected solver output	Solver proposes feasible/optimal assignment; publish step makes it active.
External calendar record	External provider + E-7 mapping	E-7 stores provider external ID/version/sync token; callbacks deduplicate.
ERP people/room/master data	ERP in Embedded mode; canonical cache/registry in E-7	Source freshness and external IDs explicitly tracked.
Audit history	E-7 audit_events + engine history refs	Append-only logical history; no ordinary user edits.
AI model output	Never authoritative	Persist only after schema/policy/graph validation and configured human approval.

6. Canonical Data Model and Persistence

PostgreSQL is the authoritative transactional boundary. Engine-specific identifiers (Flowable deployment/process/task IDs, UniTime identifiers, calendar IDs) are stored as mappings, never used as the public canonical identity. Every tenant-scoped table includes tenant_id directly or through an enforced parent relation.

Entity	Purpose / key fields
tenants	id, name, mode, settings_json, status
users	id, tenant_id, external_id, display_name, role_refs
goals	id, tenant_id, title, description, owner_id, target_date, status, context_json
plans	id, goal_id, version, source, model_id, confidence, approved_by, approved_at, baseline_hash
tasks	id, plan_id, parent_id, title, description, priority, status, effort_estimate, start/end constraints, assignee/resource refs
task_dependencies	predecessor_id, successor_id, relation_type, lag, source, confidence
milestones	id, plan_id, title, target_date, status
task_risks	id, task_id, risk_type, severity, probability, signal, mitigation, status
workflow_definitions	id, tenant_id, key, version, engine_ref, artifact_uri/hash, active
workflow_instances	id, definition_id, business_key, engine_instance_ref, canonical_state, started_at, ended_at
workflow_tasks	id, instance_id, engine_task_ref, task_id, assignee/role, state, due_at
sla_policies	id, tenant_id, name, duration, warning_offset, business_calendar_id, escalation_policy
resources	id, tenant_id, type, capacity, skills_json, external_id, provider, active
resource_availability	resource_id, start_at, end_at, availability_type, source, source_version
constraints	id, tenant_id, scope_type, scope_id, type, hard, weight, payload_json, active
schedule_jobs	id, tenant_id, scope, status, solver, solver_version, objective_json, seed/settings, started_at, finished_at
schedule_assignments	id, job_id, task/event_id, resource_id, room_id, start_at, end_at, locked, score_components
calendar_links	id, tenant_id, internal_ref, provider, external_ref, external_version, sync_token
domain_events	id, tenant_id, aggregate_type, aggregate_id, event_type, schema_version, payload, created_at, published_at
event_inbox	consumer, event_id, processed_at, outcome
audit_events	id, tenant_id, actor, action, entity_type, entity_id, before_json, after_json, source, correlation_id, created_at

6.1 Database rules

- Use UUIDs or another globally safe canonical identifier; never expose sequential cross-tenant IDs without authorization checks.

- All state-changing API transactions write the canonical change and outbox event atomically.
- Approved plans are versioned and immutable as baselines; regeneration creates a new plan version.
- Workflow artifacts are versioned and hashed; running instances remain pinned to deployed definition versions unless an explicit migration occurs.
- Published schedule assignments can be locked. Repair jobs create a new schedule version and preserve history.
- External mappings store provider + external_id + source version/sync token to prevent duplicates.
- Audit records are append-only at application level and protected from ordinary update/delete permissions.

7. State Machines

Object	Lifecycle	Key invariants
Plan	DRAFT -> GENERATED -> VALIDATED -> APPROVAL_PENDING -> APPROVED -> ACTIVE -> COMPLETED / CANCELLED	No activation before schema/policy/graph validation. Regeneration creates a new version.
Task	DRAFT -> READY -> IN_PROGRESS <-> BLOCKED -> DONE / CANCELLED	Readiness depends on predecessors and policy. Block/unblock is explicit and auditable.
Workflow definition	DRAFT -> VALIDATED -> DEPLOYED -> RETIRED	Immutable deployed version; new changes produce a new version.
Workflow instance	RUNNING -> SUSPENDED / COMPLETED / TERMINATED / FAILED	Flowable is authoritative; E-7 maps states without losing engine detail.
Schedule job	REQUESTED -> SOLVING -> FEASIBLE / INFEASIBLE -> REVIEW -> PUBLISHED -> SUPERSEDED	No publish on infeasible result. Published/locked assignments constrain repair.
Risk	OPEN -> ACKNOWLEDGED -> MITIGATING -> RESOLVED / ACCEPTED	Trigger signal and evidence remain inspectable; LLM may summarize only.

8. API and Contract Implementation

All APIs are versioned under /api/v1. Tenant scope is derived from the authenticated principal/API key, not accepted from arbitrary client fields. Mutating requests accept idempotency keys where a retry could duplicate side effects.

Endpoint	Contract
POST /goals	Create normalized operational goal.
POST /goals/{id}/generate-plan	Enqueue Task Intelligence job and create new draft plan version.
POST /plans/{id}/validate	Run schema, graph, date, policy and entity validation.
POST /plans/{id}/approve	Approve/freeze baseline and emit plan.approved.
GET /plans/{id}/critical-path	Return CPM path, slack and current risk summary.
PATCH /tasks/{id}	Update progress/status/actuals/blocker with state-transition validation.
POST /workflow-definitions	Create/import BPMN/DMN/form artifact set.
POST /workflow-definitions/{id}/deploy	Server-side validate and deploy version to Flowable.
POST /workflow-instances	Start process for canonical business object.

Endpoint	Contract
POST /workflow-tasks/{id}/actions/{action}	Approve/reject/complete only after actor eligibility check.
POST /schedule-jobs	Submit optimization job with scope, constraints and objective profile.
GET /schedule-jobs/{id}	Return state, feasibility, score, assignments and diagnostics.
POST /schedule-jobs/{id}/publish	Publish approved feasible assignments and emit calendar/event commands.
POST /schedules/{id}/repair	Create targeted repair job from disruption set.
GET /resources/availability	Query normalized availability with freshness metadata.
GET /analytics/operations	Operational KPI endpoint.
GET /audit	Authorized audit search by entity/correlation/time window.

8.1 Core typed schemas

```
GoalCreate {
  title, description, deadline?, success_criteria[], constraints[],
  resource_scope[], workflow_template?, context_refs[]
}
PlanSchema {
  phases[], tasks[{temp_id,parent_temp_id,title,description,role,effort,
            acceptance_criteria[],dependencies[],constraints[]}],
  milestones[], assumptions[], risks[], model_metadata
}
ScheduleJobRequest {
  scope_ref, horizon, resources[], demand_items[], hard_constraints[],
  soft_constraints[], objective_profile, locked_assignments[]
}
```

9. Domain Event Implementation

Event	Primary consumers
goal.created	Task Intelligence, analytics
plan.generated	Review UI, audit
plan.approved	Workflow starter, scheduler
task.blocked	Risk monitor, manager dashboard, E-8
task.completed	Workflow, milestone tracker, scheduler
workflow.task.created	Task inbox, E-8
workflow.sla.warning	E-8, manager dashboard
workflow.sla.breached	Escalation, audit
resource.unavailable	Scheduler repair queue
schedule.generated	Review UI, calendar adapter
schedule.published	Calendar sync, E-8

Event	Primary consumers
schedule.changed	Affected users, audit, analytics

9.1 Event reliability pattern

1. Write domain state and an unpublished outbox row in the same PostgreSQL transaction.
2. Outbox publisher reads unpublished rows, publishes to RabbitMQ, then marks published_at. It must be restart-safe.
3. Every event has event_id, tenant_id, event_type, schema_version, aggregate identity, occurred_at and correlation_id.
4. Consumers maintain an inbox/deduplication record by consumer + event_id before committing side effects.
5. External side effects (email command, calendar publish, ERP update) use stable idempotency keys derived from the business operation.
6. Failed consumers retry with bounded backoff. Poison events go to a dead-letter path with audit/alert, not an infinite loop.
7. Schema changes are versioned; consumers support old versions during the compatibility window.

10. Task Intelligence - Detailed Implementation

10.1 Goal-to-plan pipeline

1. Normalize request: objective, scope, tenant/domain, deadline, success criteria, constraints, allowed resources and source context.
2. Resolve organizational references before prompting: roles, resources, approved templates, policies, calendar constraints and permitted context only.
3. Optionally retrieve similar approved plans/templates. Retrieval is advisory; source/version must be attached to context.
4. Call PydanticAI with a strict PlanSchema. Use Sarvam-105B as the first benchmark candidate and at least one configured fallback model.
5. Reject any non-parseable or schema-invalid output; never persist malformed model output as a plan.
6. Run deterministic validators: required fields, stable temporary IDs, valid parent refs, duplicate detection, entity resolution, impossible dates, dependency cycles and policy rules.
7. If validation fails, pass only machine-readable validation errors back to the planner for bounded targeted repair. Limit retry count.
8. Compute NetworkX DAG, topological order, CPM/slack and critical path. Attach graph diagnostics to plan version.
9. Compute transparent initial effort estimates from heuristics/historical medians. Do not promote ML estimation until clean execution history exists.
10. Compute confidence/quality signals and route to human review according to policy. Human edits create an audit trail.
11. On approval, freeze baseline, materialize canonical tasks/dependencies/milestones, emit plan.approved and trigger configured downstream workflow/scheduling.

10.2 Dependency and CPM rules

- Directed graph must remain acyclic before activation. On a cycle, return the exact cycle path and block activation.
- Topological order drives task readiness but does not override workflow authorization.

- CPM calculates earliest start/finish, latest start/finish, total float/slack and critical path based on current durations/calendars.
- Negative/zero slack on a blocked critical-path task increases risk severity.
- Every graph computation stores algorithm/version/input baseline hash for reproducibility where practical.

10.3 Effort estimation maturity path

Stage	Method	Promotion gate
Stage 0	Rules/standard durations by task type and role	Available immediately; transparent and editable.
Stage 1	Historical medians/quantiles by comparable task features	Enough clean completed tasks and stable categorization.
Stage 2	XGBoost/LightGBM regression with confidence/interval calibration	Offline validation beats baseline on agreed error metric without unacceptable bias.
Stage 3	Per-domain/tenant calibration where data volume allows	Governance, drift monitoring, rollback and audit of model version.

10.4 Risk/blocker detection

- Overdue predecessor
- Negative or shrinking slack
- Capacity overload
- Repeated workflow rejection/rework
- SLA warning/breach
- Stale task/no progress
- Estimate vs actual variance
- Resource absence
- Room/equipment loss
- Dependency-chain growth
- Schedule instability after repair

Risk triggers remain rules/statistical signals with inspectable evidence. An LLM may summarize the situation or propose mitigations, but cannot create the underlying authoritative risk signal without a traceable trigger.

11. Workflow Mode - Detailed Implementation

11.1 Flowable architecture

1. Run Flowable Engine as a dedicated Java service with its own engine schema/database access according to deployment choice; keep E-7 canonical mappings in the main PostgreSQL schema.
2. Embed bpmn-js, dmn-js and form-js in the IR Operations UI. The UI edits artifacts; it does not directly deploy to the engine.
3. On save, version BPMN/DMN/form artifacts in E-7, validate syntax and policy server-side, compute artifact hashes, and require appropriate designer/admin permissions.
4. On deploy, call Flowable REST/Java integration through the workflow adapter and store deployment/process definition references on the E-7 workflow_definition version.
5. When a plan/task triggers a workflow, create canonical workflow_instance, start Flowable with a stable business key/correlation ID, and store the engine instance reference.
6. Map Flowable user tasks to canonical workflow_tasks. All user actions go through E-7 authorization, then invoke the exact engine action.

7. Use Flowable DMN for deterministic branching decisions and persist rule key/version/inputs/output in audit metadata.
8. Use timers for SLA warning/breach semantics; business-calendar calculations are controlled by the E-7 SLA policy mapping.
9. Publish workflow task/SLA/completion events via the canonical event gateway rather than letting each BPMN service task send notifications directly.
10. Keep Temporal absent from the MVP. Add it only when a code-heavy durable subprocess has a documented Flowable limitation and ADR approval.

11.2 Supported workflow primitives

- Start/end events
- Human user task
- Service task
- Timer event
- Message/event catch
- Exclusive gateway
- Parallel gateway
- Inclusive gateway
- Subprocess/call activity
- Versioned typed process variables

11.3 Approval models

- Single approver
- Sequential hierarchy
- Parallel all-of
- Quorum N-of-M
- Role-based pool
- Delegated approval
- Escalated approval

11.4 SLA policy schema

```
SLA Policy:
- start_trigger
- duration / business_time_unit
- business_calendar_id
- pause_states[]
- warning_offset
- breach_action
- escalation_chain[]
- reassignment_policy
- notification_policy_ref
```

> **Important:** Persisted workflow/timer state must survive worker/service restarts. Do not implement authoritative SLA timers as an in-memory FastAPI scheduler.

12. Scheduling and Resource Optimization - Detailed Implementation

12.1 Solver authority and profiles

OR-Tools CP-SAT is the primary generic solver. UniTime is evaluated for education-specific scheduling using the same canonical benchmark data. The production path is chosen by Phase 5 gate. FET and Timefold remain benchmark/conditional alternatives and are not simultaneous MVP dependencies.

12.2 Hard constraints

- No person/room/equipment double booking
- Resource availability and blackout periods
- Room capacity/type/equipment requirements
- Faculty qualification or role eligibility
- Precedence and minimum-gap rules
- Duration and allowed time windows
- Locked/published assignments that cannot move
- Tenant/domain policy constraints

12.3 Soft constraints

- Minimize idle gaps and room changes
- Prefer requested time/resource choices
- Balance workload fairly
- Minimize changes during repair
- Spread exams/critical activities with preferred gaps
- Maximize utilization while preserving buffers

12.4 Scheduling job flow

1. Resolve canonical demand items, resources, availabilities, constraints and current locked assignments for a versioned scheduling scope.
2. Freeze an input snapshot/hash so the solve is reproducible and stale-result detection is possible.
3. Build solver model and objective weights from a named solver profile. Hard constraints are mandatory; soft constraints become weighted penalties/rewards.
4. Run in an isolated worker with a per-job time budget. Persist solver name/version/settings/seed and progress metadata.
5. If feasible, persist candidate assignments and score components as REVIEW state. Do not publish automatically unless a tenant policy explicitly allows it.
6. If infeasible, return explicit diagnostic constraints/resource conflicts and suggested relaxation candidates. Never return an empty schedule as silent success.
7. On human/system-approved publish, create a new schedule version, lock configured assignments, emit schedule.published and invoke calendar adapter.
8. On disruption, construct a repair job for affected neighborhood only, preserve locked/unaffected assignments, add movement penalties, and publish a superseding version after review.

12.5 UniTime PoC boundary

- Treat UniTime as an education scheduling service behind an adapter, not as the E-7 canonical database.
- Map academic sessions, rooms, staff, courses/exams/events, availability, preferences and external IDs from E-7/ERP to UniTime data-exchange interfaces.
- Run identical datasets through UniTime and an OR-Tools-native model. Compare feasibility, solve time, soft violations, repair stability, adapter effort and operations burden.
- Adopt UniTime only if it materially reduces educational constraint implementation/maintenance while preserving E-7 API and canonical model boundaries.

13. Resource, Calendar and Integration Services

13.1 Resource service

- Normalize people, rooms, equipment and other resources into one canonical resource model.

- Store skills/capabilities, capacities, active status, provider/external_id and availability freshness.
- Separate resource identity from availability intervals.
- Every recommendation checks eligibility + availability + tenant policy before ranking.
- Resource data originating in an ERP keeps the ERP as master where configured; E-7 records mapping and freshness.

13.2 Calendar adapter

- Use provider-specific adapters behind one canonical interface.
- Normalize time zones before comparison/solve.
- Maintain external event ID, version/etag where available and sync token.
- Use idempotent create/update/delete operations.
- Deduplicate callbacks using external ID/version + internal idempotency key.
- Published schedule changes create auditable calendar-sync status rather than pretending success during provider outage.

13.3 Other unified-engine integrations

Engine	E-7 integration
E-1 Knowledge & Content	Retrieve approved SOPs/policies/templates as constrained planning context.
E-4 Intelligence & Insights	Consume demand/forecast signals; provide operations telemetry for analytics.
E-6 Observation	Receive operational/physical events such as resource unavailable that can trigger workflow or repair.
E-8 Engagement	Deliver approval reminders, SLA warnings/breaches and schedule-change notifications.
E-9 People Intelligence	Consume authorized staff skills/roles/availability for resource matching.

14. Security, Tenancy and Authorization

Control	Implementation requirement
Authentication	OIDC/JWT for users; scoped API keys/service identities for machine clients. Validate issuer, audience, expiry and key rotation.
Tenant resolution	Derive tenant from authenticated principal/API key. Never trust arbitrary tenant_id request fields.
Authorization	RBAC for coarse roles; ABAC/policy checks for department/object/current workflow task/context restrictions.
Workflow actions	Verify actor is eligible for the exact current task and requested transition before calling Flowable.
Model privacy	Send minimum necessary context. Never insert secrets/unrestricted DB data. Redact or tokenize sensitive identifiers where possible.
Model trust	Treat every model output as untrusted input until schema/entity/policy/graph validation completes.
Webhooks	Signed requests, timestamp/replay protection, deduplication and least-privilege callback credentials.

Control	Implementation requirement
Audit	Append-only logical history for state changes, model recommendations, rule outcomes, solver jobs and external side effects.
Data lifecycle	Tenant-configurable retention/export/delete policy compatible with customer contracts and applicable law.
Secrets	Secret manager/environment injection; never commit credentials; rotation support.

15. Reliability, Idempotency and Failure Recovery

Failure	Required behavior
LLM timeout / invalid JSON	Bounded retry; configured fallback model; persist job failure, never malformed plan.
Hallucinated resource/entity	Reject unresolved references before canonical persistence.
Dependency cycle	Block activation; return cycle path and repair hint.
Flowable/worker crash	Durable runtime resumes; external service actions are idempotent.
ERP unavailable	Retry/backoff + degraded integration state; no fake success.
Solver timeout	Return best feasible solution only if policy permits; mark optimality unknown; otherwise unresolved with diagnostics.
No feasible schedule	INFEASIBLE + conflict explanation + candidate relaxations; human decides.
Calendar duplicate callback	Deduplicate by provider/version + idempotency key.
Broker outage	Persist outbox; publish when broker recovers.
Cross-tenant reference	Reject before data access/state change and emit security audit event.

16. Observability and Operational Metrics

Metric family	Minimum telemetry
Planning	Generation latency, model/provider, tokens/cost, structured-validity rate, validation failures, human correction rate, accepted-vs-edited task ratio.
Workflow	Instances started/completed/failed, user-task queue age, SLA warning/breach rate, retries, stuck instances, engine/API latency.
Scheduling	Solve duration, feasibility rate, objective score, hard/soft violations, repair movement count, solver timeout rate.
Resources	Utilization, over-allocation attempts, availability freshness, unresolved external references.
Reliability	API p50/p95/p99, error rate, broker lag, DB pool saturation, outbox backlog, dead-letter count.

Metric family	Minimum telemetry
Business	Approval turnaround, on-time milestone rate, blocker duration, schedule stability, manual planning time reduction.

- Every request/job/event carries a correlation_id through API, database, broker, Flowable adapter, solver and external integrations.
- Use OpenTelemetry traces; export metrics to Prometheus/Grafana or organization standard.
- Logs are structured JSON with tenant-safe fields; do not log secrets or raw sensitive model context.
- AI and solver versions/settings are recorded so regressions can be correlated with release changes.

17. Repository and Development Environment

```
operations-engine/
  app/
    api/v1/
    core/               # config, auth, tenancy, logging, idempotency
    domain/             # canonical entities, enums, state transition policies
    task_intelligence/  # planner, prompts, schemas, validators, graph, estimators
    workflow/           # Flowable adapter, DMN, SLA mappings, BPMN artifact management
    scheduling/         # constraint catalog, OR-Tools builders, solver profiles, repair
    resources/          # registry, skills, availability
    integrations/       # ERP, calendar, E-8, webhooks, UniTime adapter
    events/             # outbox, schemas, publisher, consumers, inbox
    audit/
    analytics/
    db/                 # SQLAlchemy models, repositories, Alembic migrations
  ui/                   # operations web UI; bpmn-js/dmn-js/form-js integration
  tests/
    unit/ integration/ contract/ e2e/ performance/ security/ ai_eval/
  workflows/            # versioned BPMN/DMN/form templates
  solver_profiles/
  scripts/
  docker/
  docs/
    adr/ api/ data-model/ runbooks/ user-guides/
```

17.1 Local development stack

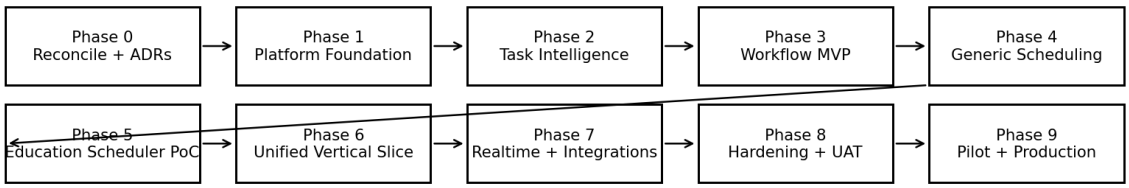
- Docker Compose first for developer/staging repeatability.
- FastAPI service + task worker + scheduler worker.
- Flowable Java service.
- PostgreSQL.
- RabbitMQ.
- Optional observability stack for local integration tests.
- Do not start Temporal, Kafka, Valkey, separate vector DB or FET/Timefold services by default.

17.2 Configuration

Variable	Meaning
DATABASE_URL	PostgreSQL connection
BROKER_URL	RabbitMQ initial endpoint
JWT_ISSUER / JWKS_URL	Identity provider
LLM_PROVIDER / MODEL	Selected planner route

Variable	Meaning
LLM_FALLBACK_MODEL	Approved fallback
LLM_TIMEOUT / MAX_RETRIES	Bounded inference resilience
FLOWABLE_BASE_URL	Flowable endpoint
UNITIME_*	PoC/production education scheduler adapter settings if selected
CALENDAR_PROVIDER_*	CalDAV/Google/Microsoft/ERP adapter config as permitted
SOLVER_MAX_SECONDS	Default solve budget
AUDIT_RETENTION_DAYS	Tenant policy default
OTEL_EXPORTER_*	Telemetry export
VALKEY_URL	Optional; absent until enabled

18. Phased Implementation Roadmap - Master Sequence



Exit gates

G0: Architecture + licenses G1: Core API + tenancy G2: Goal -> approved plan G3: Workflow restart-safe G4: Zero hard conflicts

G5: Scheduler selection G6: End-to-end audit G7: Adapter + repair G8: Production readiness G9: Pilot sign-off

Conditional components (Temporal, LangGraph, Timefold, FET, Valkey, separate vector DB) are introduced only after a measured need and ADR approval.

Phase	Name	Outcome
0	Reconciliation, PoC baselines & ADR freeze	Source traceability, license screen, datasets, provider/ERP discovery, final interfaces.
1	Platform foundation	FastAPI, auth/tenancy, PostgreSQL/Alembic, audit, outbox, RabbitMQ, CI/OpenAPI.
2	Task Intelligence MVP	Goal -> typed WBS -> validation -> NetworkX/CPM -> human approval.
3	Workflow MVP	bpmn-js/DMN/form authoring, Flowable deployment/runtime, approvals, SLA, audit.

Phase	Name	Outcome
4	Generic Scheduling MVP	OR-Tools resource/availability/constraints, feasibility, publish/lock/repair basics.
5	Education Scheduling PoC	UniTime vs OR-Tools-native vs FET benchmark; select production education path.
6	Unified vertical slice	Approved plan -> workflow -> scheduling -> calendar/event loop -> audit.
7	Real-time operations & integrations	Risk monitor, resource disruptions, incremental repair, ERP/calendar/E-8 adapters.
8	Hardening & production readiness	Security, performance, HA/recovery, observability, UAT, docs, release automation.
9	Pilot & production rollout	Controlled tenant rollout, monitoring, feedback, rollback, acceptance sign-off.
10	Post-production optimization	Historical effort ML, optional LangGraph/Temporal/Timefold/Valkey based on evidence.

Phase 0 - Reconciliation, Baselines and Architecture Freeze

Objective: Turn all supplied documents and OSS research into executable decisions before coding, with no ambiguous ownership or hidden license risk.

Implementation steps

1. Create a requirements traceability matrix for FR-TI-01..09, FR-WF-01..07, FR-SC-01..06 and FR-INT-01..03. Map each requirement to service, database entity, API, test and acceptance evidence.
2. Freeze source precedence rules from Section 1 and publish ADR-000 Source-of-Truth and Reconciliation.
3. Publish ADRs for: Flowable primary / Operaton fallback; PydanticAI MVP / LangGraph conditional; OR-Tools generic primary; UniTime education PoC; Flowable DMN before Drools; RabbitMQ initial broker; PostgreSQL canonical store; Valkey optional.
4. Run license review for Apache/MIT/BSD/MPL/bpmn.io special terms and AGPL benchmark components. Record allowed deployment/distribution pattern for each.
5. Define one representative vertical use case (recommended: semester examination operations) and one non-education generic scheduling use case.
6. Prepare fixed PoC datasets: gold-set goals and expected plan properties; workflow scenarios; feasible/infeasible/repair scheduling datasets; resource/ERP fixture data.
7. Define exact identity provider, first ERP/SIS adapter, first calendar provider and E-8 notification contract or mock.
8. Define production SLO candidates, data retention assumptions and first-tenant scale envelope for benchmarking.

Deliverables

- Traceability matrix
- ADR set
- License/risk matrix

- PoC datasets
- Benchmark metric definitions
- Integration contract inventory
- Project backlog/epics

Exit gate: No unresolved blocker on primary stack; datasets and ADRs approved; every source requirement mapped.

Phase 1 - Platform Foundation and Canonical Core

Objective: Create the secure, tenant-aware transactional skeleton shared by all three modes.

Implementation steps

1. Create repository, Python environment, lint/type/test tooling, branch protections and CI skeleton.
2. Implement configuration management, structured logging, correlation IDs and health/readiness endpoints.
3. Implement OIDC/JWT + service API key authentication; derive tenant from principal and create RBAC hooks.
4. Define canonical SQLAlchemy models and Alembic migrations for tenants/users/goals/plans/tasks/dependencies/milestones/resources/constraints/schedule jobs/events/audit.
5. Implement repository/unit-of-work layer and transaction boundaries.
6. Implement append-only audit helper and actor/source/correlation metadata.
7. Implement transactional outbox and RabbitMQ publisher; define event envelope v1 and consumer inbox/deduplication.
8. Implement idempotency-key middleware/store for mutating APIs and external side effects.
9. Create base /api/v1 routers, OpenAPI conventions, standard error model and job-status model.
10. Build Docker Compose for API, worker, PostgreSQL, RabbitMQ and Flowable placeholder; add migration/startup scripts.
11. Add OpenTelemetry instrumentation and baseline metrics.

Deliverables

- Running core API
- Database migrations
- Auth/tenant isolation
- Audit/outbox
- RabbitMQ event path
- CI + Docker Compose
- OpenAPI baseline

Exit gate: Core API deployed in dev/staging; cross-tenant negative tests pass; state change + outbox atomicity proven.

Phase 2 - Task Intelligence MVP

Objective: Deliver reliable goal -> approved executable plan while keeping the model non-authoritative.

Implementation steps

1. Define Pydantic GoalCreate, PlanSchema, TaskDraft, DependencyDraft, MilestoneDraft and validation-error schemas.
2. Implement model-provider abstraction and PydanticAI planner with Sarvam-105B benchmark route plus at least one approved fallback.

3. Implement context builder that retrieves only authorized policies/templates/resource metadata and records sources.
4. Implement strict structured output parsing, timeout/retry/fallback, model metadata and cost/latency telemetry.
5. Implement validation pipeline: schema, temporary IDs, parent refs, duplicates, dates, resource/entity resolution, policy checks.
6. Implement NetworkX DAG, cycle path, topological ordering and deterministic CPM/slack service.
7. Implement bounded targeted repair using machine-readable validation errors; cap repair attempts.
8. Implement transparent effort heuristics/historical median interface; leave XGBoost/LightGBM behind a future feature flag.
9. Implement plan versioning, review/edit UI/API, approval/freeze baseline and plan.generated/plan.approved events.
10. Create AI gold-set evaluation harness and regression report for structured validity, required-task coverage, invalid dependencies, hallucinated entities and human correction rate.

Deliverables

- Goal intake
- Planner worker
- PlanSchema + validators
- NetworkX/CPM
- Plan review/approval
- AI evaluation report

Exit gate: Representative goals consistently produce schema-valid, cycle-free plans and approved baselines; model cannot bypass validators.

Phase 3 - Workflow MVP

Objective: Execute institutional approval/business processes with deterministic state, persisted timers and audit.

Implementation steps

1. Deploy Flowable service with controlled database/configuration and secure service-to-service authentication.
2. Implement workflow adapter interface and Flowable adapter: deploy definition, start instance, query state, fetch user tasks, complete action, history mapping.
3. Embed bpmn-js in Operations UI; add server-side BPMN validation and versioned artifact storage.
4. Embed dmn-js and Flowable DMN for at least one typed branching decision; persist rule version/inputs/output.
5. Embed form-js or IR form mapping for human task forms; bind allowed typed variables only.
6. Implement canonical workflow_definition/instance/task mappings and state synchronization.
7. Implement actor eligibility checks for approve/reject/complete endpoints.
8. Implement SLA policies, timer mappings, warning/breach events, pause/resume semantics and escalation configuration.
9. Implement idempotent service-task callback pattern; prevent duplicate ERP/notification side effects on retries.
10. Test sequential, parallel, quorum/rejection, timer/SLA, worker restart, duplicate callback and unauthorized-action scenarios.

Deliverables

- Workflow designer
- Flowable adapter
- DMN rule path
- Human task inbox/actions
- SLA/escalation
- Workflow audit

Exit gate: Representative workflows pass E2E + restart/retry tests without duplicate side effects.

Phase 4 - Generic Scheduling MVP

Objective: Generate explainable conflict-free schedules and controlled repairs using OR-Tools CP-SAT.

Implementation steps

1. Implement resource registry/availability APIs and constraint catalog model.
2. Create solver profile schema for hard constraints, soft weights, horizon, time granularity and max solve time.
3. Implement OR-Tools builder modules for no-overlap, capacity, eligibility, precedence, time-window, gap and lock constraints.
4. Implement weighted soft objectives for preferences, idle gaps, fairness, utilization and movement penalty.
5. Implement schedule job queue/worker, input snapshot hash, solver metadata/version/settings and status API.
6. Persist candidate assignments and score components; keep publish as separate authorized action.
7. Implement infeasibility diagnostics by mapping model constraints back to canonical constraint IDs and human-readable explanations.
8. Implement lock/publish and initial repair API with affected-scope + movement penalties.
9. Implement schedule.generated/published/changed events and calendar adapter command interface.
10. Run feasible, infeasible, stress and repair regression datasets.

Deliverables

- OR-Tools scheduling service
- Constraint catalog
- Publish/lock
- Conflict explanation
- Repair basics
- Regression suite

Exit gate: Zero hard-constraint violations on feasible benchmarks; explicit diagnostics on infeasible cases.

Phase 5 - Education Scheduling PoC and Production Choice

Objective: Choose the education-specific scheduling implementation without contradicting the generic solver architecture.

Implementation steps

1. Build one canonical school/exam dataset containing sessions, student groups, subjects, rooms, invigilators, availability, capacities, equipment and gap/preferences.
2. Implement minimal UniTime adapter/import-export mapping using external IDs; do not move canonical authority into UniTime.
3. Implement equivalent OR-Tools-native academic constraint profile on the same data.
4. Run FET only as benchmark/fallback under approved legal/service-boundary conditions.

5. Measure feasibility, solve duration, soft violations, schedule quality, incremental-change stability, adapter complexity, deployment/ops cost and explainability.
6. Document edge cases: room outage, faculty absence, exam gap, fixed event, locked published assignment and partial repair.
7. Choose one production education path: UniTime service behind adapter or OR-Tools-native model. Keep the other as optional/reference unless a dual path has a strong business reason.
8. Publish ADR with benchmark data and licensing decision.

Deliverables

- UniTime adapter PoC
- OR-Tools academic profile
- FET benchmark evidence if approved
- Benchmark report
- Production scheduler ADR

Exit gate: Education scheduler path selected with measurable evidence; no hard conflicts; repair behavior accepted.

Phase 6 - Unified Vertical Slice

Objective: Prove the complete E-7 Plan -> Automate -> Schedule -> Monitor loop with one real operational scenario.

Implementation steps

1. Use semester examination operations as reference vertical slice or another approved business flow.
2. Goal creates a typed plan; dependency/CPM validators pass; admin approves baseline.
3. plan.approved starts the configured Flowable process and creates readiness/approval tasks.
4. Workflow completion/events create schedule demand items and/or unlock scheduling milestones.
5. Scheduler generates candidate timetable/resource allocations; scheduler admin reviews and publishes.
6. Calendar adapter syncs published assignments; E-8/mock receives notification commands.
7. Simulate resource.unavailable; event consumer creates targeted repair preserving locked/unaffected assignments.
8. Correlate all API calls, model versions, rule results, workflow transitions, solver job, schedule change and notifications in audit search.
9. Expose operations dashboard showing plan progress, blockers, SLA state, schedule health and recent events.

Deliverables

- End-to-end production-shaped slice
- Cross-service audit
- Demo/UAT script
- Operations dashboard MVP

Exit gate: One goal runs through all three modes and a disruption repair with complete audit and no manual database edits.

Phase 7 - Real-Time Operations, Risk Monitor and External Integrations

Objective: Make E-7 operational against real system events instead of only request/response demos.

Implementation steps

1. Implement scheduled/event-triggered risk monitor for critical path, stale task, capacity, SLA and schedule instability signals.
2. Implement first real ERP/SIS adapter using canonical external IDs and contract tests.
3. Implement first real calendar provider adapter with incremental sync, dedupe and rollback behavior.
4. Integrate E-8 notification contract or chosen messaging boundary for workflow/SLA/schedule changes.
5. Implement resource freshness tracking and resource.unavailable/change consumers.
6. Implement controlled incremental rescheduling queue and movement/stability KPIs.
7. Add operational analytics endpoints and dashboards for throughput, blockers, SLA, utilization, feasibility and repair movement.
8. Exercise broker outage, ERP outage, calendar callback replay and worker restart scenarios.

Deliverables

- Risk monitor
- ERP adapter
- Calendar adapter
- E-8 integration
- Realtime repair
- Operational analytics

Exit gate: External event changes reliably propagate through canonical events and repair/notifications with idempotent recovery.

Phase 8 - Security, Performance, Recovery and UAT Hardening

Objective: Meet production readiness before any customer pilot.

Implementation steps

1. Run authorization matrix and cross-tenant penetration tests, BOLA/object access tests, role escalation tests and webhook replay tests.
2. Run prompt injection/secret leakage tests against planner/context builder and enforce context minimization/redaction.
3. Load test CRUD APIs, plan jobs, workflow concurrency/timers, solver queues and event consumers against target tenant scale.
4. Validate PostgreSQL backup/PITR, broker recovery, Flowable recovery, worker restart, outbox replay and calendar/ERP retry runbooks.
5. Tune DB indexes, worker pools, queue limits, solver time budgets and tenant quotas based on evidence.
6. Complete OpenAPI, ERD, workflow authoring guide, constraint catalog, model evaluation guide, deployment runbook and incident guide.
7. Execute UAT scripts for planner, approver, scheduler, auditor and tenant admin roles.
8. Freeze release candidate, SBOM/dependency scan, SAST, secret scan and container scan.

Deliverables

- Security report
- Performance report
- Recovery evidence
- Runbooks
- UAT evidence

- Production checklist

Exit gate: No unresolved critical/high security defect; recovery and UAT pass; production readiness sign-off.

Phase 9 - Pilot and Production Rollout

Objective: Deploy safely to a controlled tenant and prove operational value.

Implementation steps

1. Deploy production infrastructure using Docker/Kubernetes strategy appropriate to measured scale; keep architecture portable.
2. Migrate/read-only sync first tenant data into staging; validate identifiers, time zones, resources and workflow mappings.
3. Enable incremental sync/webhooks; reconcile counts and sample records with business owners.
4. Perform canary/pilot rollout with rollback plan. Start with one workflow/scheduling domain, not the entire institution.
5. Monitor latency/error/SLA/solver/model/business KPIs and compare against baseline manual process.
6. Collect user corrections, schedule overrides and workflow exceptions as structured feedback.
7. Hold pilot review and either expand tenant scope, remediate issues or roll back without deleting source-system data.

Deliverables

- Production deployment
- Pilot tenant onboarding
- Monitoring baseline
- Pilot acceptance report
- Rollback evidence

Exit gate: Pilot business owner sign-off and stable operational SLOs for agreed scope.

Phase 10 - Post-Production Optimization and Conditional Components

Objective: Add sophistication only after production evidence demonstrates value.

Implementation steps

1. Collect clean actual effort/duration data and evaluate XGBoost/LightGBM against heuristic baseline before promotion.
2. Add LangGraph only if the Task Intelligence workflow requires persistent multi-agent/critic/reviewer graphs beyond the typed PydanticAI flow.
3. Add Temporal only for code-heavy long-running subprocesses that are operationally awkward in Flowable.
4. Adopt Timefold only if benchmarked Java constraint modeling materially improves maintainability/performance for a real use case.
5. Add Valkey only if cache/coordination metrics justify it; add separate vector DB only if PostgreSQL/pgvector is demonstrably insufficient.
6. Expand template catalog, solver profiles, ERP/calendar adapters and domain configurations using the same canonical contracts.

Deliverables

- Evidence-based optimization ADRs
- Optional component PoCs

- Model/solver upgrades with regression gates

Exit gate: Every added component has measurable benefit, licensing approval, operational owner and rollback path.

29. CI/CD and Release Controls

Stage	Required controls
Pull request	Formatter/lint, type checking, unit tests, API/schema compatibility, migration check, license/dependency policy.
Integration pipeline	Ephemeral PostgreSQL + RabbitMQ + Flowable; solver datasets; outbox/inbox and adapter contract tests.
AI evaluation gate	Gold-set regression must not fall below approved thresholds for structured validity/coverage/dependency quality.
Security	SAST, dependency/SBOM scan, secret scan, container scan and policy checks.
Staging	Representative workflows/schedules, integration fixtures, migration rehearsal, load smoke and recovery drills.
Production	Backward-compatible migrations, canary/rolling deployment, health/readiness gates, metrics watch and rollback.

30. Testing and Verification Matrix

Test layer	Minimum scope
Unit	Schemas, state transitions, tenant guards, dependency cycles, CPM/slack, risk rules, SLA calculations, constraint builders, scoring, adapters.
Integration	PostgreSQL transactions, outbox/broker, Flowable deploy/execute, scheduler worker, calendar/ERP adapters, idempotent external calls.
Contract	ERP/SIS, E-8 notification contract, calendar provider, event schemas, API backward compatibility.
AI evaluation	Gold-set plan properties: structured validity, task coverage, invalid dependencies, hallucinated references, correction rate, latency/cost.
Workflow E2E	Happy path, rejection, sequential/parallel/quorum approvals, SLA warning/breach, retries, restart, duplicate callback, permission denial.
Scheduling	Feasible/infeasible benchmarks, soft objectives, stress, timeout, lock/publish, absence/outage repair and stability.
Security	Cross-tenant isolation, BOLA, role escalation, webhook replay, prompt injection, secret leakage, unsafe expression injection.
Performance	API p95, workflow concurrency/timers, event throughput/backlog, solver queue/solve budget, DB pool, model latency.
Recovery	DB restore/PITR, broker outage/outbox replay, worker crash, Flowable restart, provider outage, duplicate callbacks.

Test layer	Minimum scope
UAT	Planner/manager, workflow designer, approver, scheduler, auditor, tenant admin and ERP client journeys.

31. Consolidated Acceptance Criteria

Area	Acceptance criterion
Task generation	Valid work items can be generated and persisted only after schema validation.
Dependencies	No activated plan contains cycles; dependency relationships and critical path are explainable.
Resource recommendation	When eligible resources exist, ranked feasible suggestions include reasons/conflicts.
Progress	Task updates automatically update project/milestone health.
Risk/blocker	Known regression scenarios trigger configured signal with inspectable evidence.
Workflow	Sequential/parallel configured approval chains execute correctly.
Rules	Same typed inputs + rule version produce the same result.
SLA	Warning/breach events fire at configured business-clock thresholds and survive restarts.
Audit	Every state transition and external side effect has actor/time/source/correlation metadata.
Scheduling	Feasible regression datasets have zero hard-constraint conflicts.
Infeasibility	Infeasible inputs return violated/conflicting constraints and suggested relaxations, not silent partial success.
Repair	Locked/unaffected assignments are preserved where possible and movement is measurable.
Calendar	Published changes sync idempotently and retain external mapping.
APIs	Documented success/error contract tests pass.
Recovery	Workflow/job state survives service restart without duplicate committed side effects.
Tenancy	Cross-tenant references are denied before data disclosure or state change.

32. Reference End-to-End Scenario - Semester Examinations

1. Admin submits: plan and conduct Grade 10 semester exams in the specified window; major subjects require a gap; rooms/invigilators come from ERP.
2. Task Intelligence generates phases for planning, paper readiness, approvals, room/invigilator preparation, scheduling, execution, incident handling and closure.
3. Pydantic/NetworkX validators reject invalid references/cycles and compute critical path. Admin edits/approves baseline.

4. plan.approved starts Flowable question-paper/readiness workflows; parallel department approvals use DMN where appropriate; SLA timers escalate overdue work.
5. Scheduling service resolves subjects, groups, rooms, invigilators, durations, unavailable periods and gap rules into hard/soft constraints.
6. OR-Tools or selected education scheduler returns feasible exam timetable and ranked assignments; admin reviews/publishes.
7. Calendar events synchronize and E-8 sends approved notifications.
8. An invigilator becomes unavailable; resource.unavailable triggers targeted repair while preserving locked exam times/unaffected sessions.
9. All approvals, rule outcomes, schedule scores, repair reasons, calendar mappings and notification commands remain auditable; milestones close as work completes.

33. Migration and Tenant Onboarding

1. Inventory source users, roles, projects/tasks if relevant, rooms, staff, calendars, workflow states and identifiers.
2. Define canonical mapping and immutable provider/external_id fields.
3. Perform read-only import into staging and validate referential integrity, time zones and source freshness.
4. Backfill availability and active process/schedule state only where source semantics are unambiguous.
5. Reconcile counts and representative records with business owners.
6. Enable incremental sync/webhooks before cutover; run in observe/shadow mode where possible.
7. Maintain rollback and do not delete source-system data during first production release.
8. After pilot sign-off, gradually expand write-back/publish permissions by domain.

34. Risks, Controls and Explicit Future Decisions

Risk	Control
Overusing LLMs	Typed tools + validation; deterministic rules/solvers remain authoritative.
Insufficient effort data	Heuristics/medians first; ML only after clean history and benchmark.
Licensing incompatibility	Legal review; keep AGPL/reference projects outside proprietary core unless approved.
Complex ERP schemas	Canonical model + adapter layer + contract tests.
Workflow/scheduler divergence	Shared canonical task/resource IDs and events; one authoritative owner per state domain.
Schedule instability	Lock/freeze published assignments and penalize movement during repair.
Tenant/model data leakage	Context minimization, provider policy, redaction, per-tenant configuration and self-host/open path.
Too many tools	Minimum stack first; conditional components require evidence + ADR.
Vendor/project roadmap change	Adapter boundaries for workflow/solver/model/provider; no project-specific IDs in public contracts.

Risk	Control
Bad AI plan accepted by human	Quality warnings, confidence/evidence, required review thresholds and immutable baseline/version history.

34.1 Decisions that must remain ADR-controlled

- Flowable-only vs Flowable + Temporal
- RabbitMQ vs Kafka if platform standard changes
- Sarvam-105B benchmark result and fallback model
- UniTime vs OR-Tools-native education production path
- Whether/when Timefold is justified
- Whether/when LangGraph is justified
- Valkey or separate vector DB introduction
- First identity provider/RBAC-ABAC contract
- Calendar provider set
- Production SLO/retention values
- Any AGPL component embedding/distribution pattern

35. Definition of Done

- Feature maps to a documented functional requirement and acceptance test.
- Authorization and tenant isolation are implemented and tested.
- API/event/schema is versioned and documented.
- State changes, model recommendations, rule decisions and solver jobs are auditable.
- Failure, retry and idempotency behavior is tested.
- Unit + relevant integration/E2E/security tests pass.
- Observability metrics/logs/traces are present and correlate by request/job/event.
- Documentation, migrations and runbooks are updated.
- No unresolved critical/high security defect.
- UAT evidence exists for production release scope.

36. Implementation Backlog and Suggested Epics

Epic	Scope
EPIC-CORE	Foundation, tenancy, canonical API/data model, audit/outbox/event infrastructure
EPIC-TI	Task Intelligence: planner, schemas, graph/CPM, estimates, risk, plan review
EPIC-WF	Workflow: designer, Flowable adapter, DMN/forms, approvals, SLA, audit
EPIC-SC	Scheduling: resources, constraints, OR-Tools, diagnostics, publish/repair
EPIC-EDU	UniTime/education scheduler PoC and selection
EPIC-INT	ERP, calendar, E-8 and other-engine adapters
EPIC-OPS	Observability, analytics, runbooks, reliability
EPIC-SEC	AuthN/AuthZ, tenancy, security testing, data lifecycle
EPIC-REL	CI/CD, deployment, backup/recovery, performance and UAT

36.1 Initial user stories retained from the specification

- OPS-001 Operations manager submits a goal and receives editable WBS plan.
- OPS-002 Manager approves a plan version and freezes its baseline.
- OPS-003 Project owner sees dependencies, critical path and blockers.
- OPS-004 Workflow designer deploys a versioned approval workflow.
- OPS-005 Approver acts only on tasks for which they are authorized.
- OPS-006 Admin defines SLA warning/escalation policies.
- OPS-007 Scheduler defines resources and hard/soft constraints.
- OPS-008 Scheduler generates a conflict-free timetable or receives infeasibility reasons.
- OPS-009 Scheduler locks published assignments and repairs only affected items.
- OPS-010 Auditor reconstructs who changed what, when and why.
- OPS-011 ERP integrates through idempotent APIs and domain events.
- OPS-012 Tenant admin configures behavior without exposing another tenant.

37. Final Implementation Decision

Approved architecture statement: Implement E-7 as a thin IR-owned, platform-agnostic operations intelligence layer. Reuse Flowable for workflow execution, OR-Tools for generic optimization, evaluate UniTime for education scheduling, use PydanticAI for typed task-intelligence orchestration, and NetworkX for deterministic dependency analysis. PostgreSQL remains authoritative, RabbitMQ is the initial event broker, and every external runtime stays behind an adapter. Conditional technologies are introduced only after a measured requirement and ADR approval.

This preserves every source-defined behavior while resolving older candidate-stack ambiguity in favor of the later open-source research. It also maintains IR Infotech's core philosophy: Open-Source AI First, build the intelligence layer rather than another full-stack application, remain platform-agnostic, use deterministic/auditable execution for business-critical state, and optimize for low operational complexity.

Appendix A - Traceability Summary

Area	Source requirements	Implementation mapping
Task Intelligence	WBS, dependencies, effort, allocation, milestones, progress, blockers, recommendations	PydanticAI/Sarvam benchmark, Pydantic schemas, NetworkX/CPM, heuristics -> ML, Plan Service/Risk Monitor
Workflow	No-code design, approvals, rules, SLA, escalation, automation, audit	bpmn-js/dmn-js/form-js, Flowable BPMN/DMN, E-7 auth/SLA/event/audit mappings
Scheduling	Timetables, resources, exams/events, calendar, conflict detection, optimization	OR-Tools CP-SAT, UniTime PoC, FET benchmark, Resource/Calendar services
Integration	Notifications, ERP adapters, events	E-8/adapters, canonical external IDs, RabbitMQ outbox/inbox
Security/operations	Tenant isolation, auditable, scalable/recoverable	OIDC/JWT/API keys, RBAC/ABAC, PostgreSQL, OTel, CI/CD/recovery tests

Appendix B - Source References

- [S01] IR MAIN DOC(10).pdf - mandatory research method, platform principles, original engine requirements.
- [S02] Consolidated Engine Directory(5).pdf - formal E-7 merge and Plan -> Automate -> Schedule logic.
- [S03] Engine Architecture - IR Infotech AI Ecosystem(4).pdf - guiding philosophy, dual access modes and deterministic scheduling boundary.

- [S04] 2task_intelligence(1).pdf - Task Intelligence story, deliverables and acceptance criteria.
- [S05] 13workflow(1).pdf - Workflow story, deliverables and acceptance criteria.
- [S06] 19scheduling(1).pdf - Scheduling story, deliverables and acceptance criteria.
- [S07] E-7 Comprehensive Implementation Plan - original candidate technologies and reasoning/execution separation.
- [S08] IR_Infotech_E7_Operations_Engine_Complete_Specification(3).pdf - complete reconciled E-7 functional/architecture implementation blueprint.
- [S09] IR_Infotech_E7_Operations_Engine_Open_Source_Research_Report.pdf - final OSS selection, source links, licensing and PoC decisions.

Appendix C - Official Open-Source Links

Flowable Engine: https://github.com/flowable/flowable-engine

bpmn-js: https://github.com/bpmn-io/bpmn-js

dmn-js: https://github.com/bpmn-io/dmn-js

form-js: https://github.com/bpmn-io/form-js

Google OR-Tools: https://github.com/google/or-tools

UniTime: https://github.com/UniTime/unitime

PydanticAI: https://github.com/pydantic/pydantic-ai

NetworkX: https://github.com/networkx/networkx

Temporal: https://github.com/temporalio/temporal

Timefold Solver: https://github.com/TimefoldAI/timefold-solver

LangGraph: https://github.com/langchain-ai/langgraph

FET source archive: https://lalescu.ro/liviu/fet/download.html

Operaton: https://github.com/operaton/operaton

OpenProject: https://github.com/opf/openproject

LibrePlan: https://github.com/LibrePlan/libreplan

Plane: https://github.com/makeplane/plane

Odoo: https://github.com/odoo/odoo

ERPNext: https://github.com/frappe/erpnext

PostgreSQL mirror: https://github.com/postgres/postgres

RabbitMQ: https://github.com/rabbitmq/rabbitmq-server

Apache Kafka: https://github.com/apache/kafka

Valkey: https://github.com/valkey-io/valkey

Sarvam-105B model: https://huggingface.co/sarvamai/sarvam-105b